UNIT - 3

Instruction Set Architecture

Lesson Structure

5.0 Objectives

After going through this unit we should be able to:

- define instruction set and the characteristics of instruction set;
- describe the element of an instruction and differentiate various types of operands;
- distinguish between types of instructions and operations performed by the instructions;
- differentiate types of instructions set on the basis of addresses in instruction sets;
- identity various addressing schemes and discuss instruction format design issues.

5.1 Introduction

The internal organization of a digital system is defined by the registers it employs and the sequence of micro-operations it performs on data stated in the registers. A digital computer is a general purpose digital system capable of executing various operations and in addition can be instructed as to what specific sequence of operations it must perform . The user of a computer can control the process by

means of programs that is a set of instruction that specify the operations, operands and the sequence in which processing has to occur.

The Instruction Set Architecture (ISA) is the part of the processor that is visible to the programmer or compiler designer. They are the parts of a processor design that need to be understood in order to write assembly language, such as the machine language instructions and registers. The ISA serves as the boundary between software and hardware.

In this unit we will define instruction, types of instructions, various addressing schemes and their importance. In addition, we will also discuss the design issues relating to instruction format.

5.2 Instruction Set Characteristics

The CPU is the heart and brain of the computer .The entire processing takes place in the Central Processing Unit (CPU). It performs calculations, issues the commands, coordinates with all other hardware components, and executes programs including the operating system. But to make CPU work, we must speak to it in binary machine language. In other words it is group of bits that tells the computer to perform specific operations. The collection of bits of a machine language are known as instructions, and its syntax is known as an instructions set.

Instruction set is the boundary where the computer designer and computer programmer see the same computer from different viewpoints. From the designer, point of view, the computer instruction set provides a functional description of a processor, that is :

(i) A detailed list of the instructions that a processor is capable of processing.

(ii) A describes of the types/locations/access methods for operands.

The common goal of computer designer is to build the hardware for implementing the machine's instructions for CPU. From the programmer's point of view, the user must understand machine or assembly language for low-level programming. Moreover, the user must be aware of the register set, instruction types and the function that each instruction performs. However, our prime focus is the programmer's viewpoint with the design of instruction set.

Instruction set is the collection of machine language instructions that a particular processor understands and executes. In other words, a set of assembly language mnemonics represents the machine code of a particular computer. Therefore, if we define all the instructions of a computer, we can say we have defined the instruction set. it should be noted here that the instructions available in

a computer are machine dependent, that is, a different processors have different instruction sets. However, a newer processor that may belong to some family may have a compatible but extended instruction set of an old processor of that family. Instructions can have different formats. the instruction format includes the following components:

- the instruction length;
- the type;
- length and position of operation codes in an instruction; and
- the number and length of operand addresses etc.

Each instructions consists of several fields. The most common fields found in instruction formats are:

Opcode : It specifies the operation (ADD, SUBTRACT, MULTIPLY, etc.) to be performed.

Operands : An address field of operand on which data processing is to be performed.

- An operand can reside in the memory or a processor register or can be incorporated within the operand field of instructions as an **immediate constant**. Therefore a mode field is needed that specifies the way the operand or its address is to be determined.

A sample instruction format is given in figure 1.

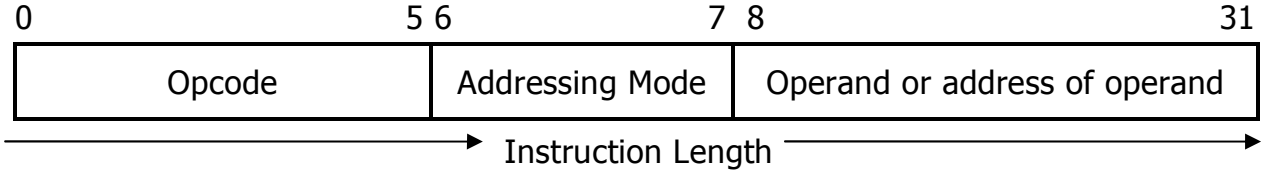


Figure 1 : An Instruction Format of 32 bits

From the above figure we have the following observations :

(i) The size of the opcode is 6 bits. So we will have $2^6 = 32$ operations.

(ii) There is only one operand address machine.

(iii) There are two bits for addressing modes. Therefore , there are $2^2 = 4$ different addressing modes possible for this machine.

(iv) The last field (8 - 31 bits = 24 bits) here is the operand or the address of operand field.

In case of immediate operand the maximum size of the unsigned operand would be 2^{24}. In case it is an address of operand in memory, then the maximum physical memory size supported by this machine is 2^{24} = 16 MB.

The opcode field of an instruction is a group of bits that define various processor operations such as LOAD, STORE, ADD, and SHIFT to be performed on some data stored in registers or memory. The operand address field can be data, or can refer to data - that is address of data, or can be labels, which may be the address of an instruction we want to execute next. Such labels are commonly used in Subroutine call instructions. An operand address can be:

- The memory address
- CPU register address
- I/O device address

The mode field of an instruction specifies a variety of alternatives for referring to operands using the given address. **It is important to know that if the operands are placed in processor registers then an instruction executes faster than that of operands placed in memory, as the registers are very highspeed memory used by the CPU. However, to put the value of a memory operand to a register we will require a register LOAD instruction.**

Instruction is represented as a sequence of bits. A layout of an instruction is termed as instruction format. Instruction formats are primarily machine dependent. A CPU instruction set can use many instruction formats at a time. Even the length of opcode varies in the same processor.

A computer can have a large number of instructions and addressing modes. The older computers with the growth of integrated circuit technology have a very large and complex set of instructions. These are called "Complex Instruction Set Computer" (CISC). Examples of CISC architectures are the Digital Equipment Corporation VAX computer and the IBM 370 computer. But it was found later that many complex instructions found in CISC are not used by the program. This lead to the idea of making a simple but faster computer, which could execute simple instructions much faster. These computers have simple instructions, registers addressing and move registers. These are called Reduced Instruction Set Computers (RISC). We will study more about RISC in unit 4 of this Block.

Instruction Set Design Considerations

Some of the basic considerations for instruction set design include selection of:

- A set of data types (e.g. integers, long integers, doubles, character strings etc.).
- A set of operations on those data types.
- A set of instruction formats. Includes issues like number of addresses, instruction length etc.

- A set of techniques for addressing data in memory or in registers.
- The number of registers which can be referenced by an instruction and how they are used.

Operand Data Types

Operand is that part of an instruction that specifies the address of the source or result, or the data itself on which the processor is to operate. Operand types usually give operand size implicitly. In general, operand data types can be divide in the following categories.

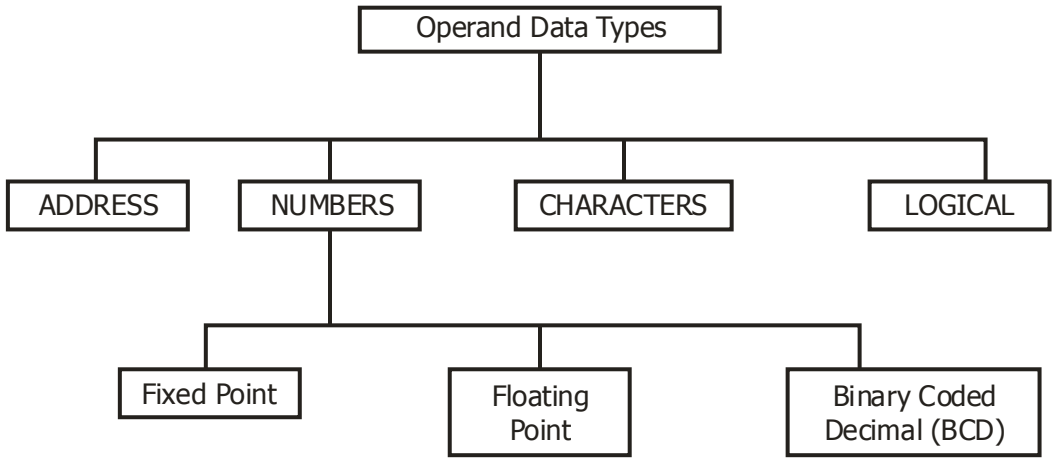


Figure 2 : Operand Data Types

- **Addresses :** Operands residing in memory are specified by their memory address and operands residing in registers are specified by a register address. Addresses provided in the instruction are operand references.
- **Numbers :** All machine languages include numeric data types. Numeric data usually use one of three representations:
- **Floating-point numbers-single precision** (1 sign bit, 8 exponent bits, 23 mantissa bits) and double precison (1 sign bit, 11 exponent bits, 52 mantissa bits).
- **Fixed point numbers (signed or unsigned).**
- **Binary Coded Decimal Numbers(BCD).**
- **Characters:** A common form of data is text or character strings. Characters are represented in numeric form, mostly in ASCII (American Standard Code for Information Exchange). Another code used to encode characters is the Extended Binary Coded Decimal Interchange Code(EBCDIC).
- **Logical data:** Each word or byte is treated as a single unit of data. When an n-bit data unit is considered as consisting of n 1-bit items of data with each item having the value 0 or 1, then they are viewed as logical data. Such bit-

oriented data can be used to store an array of Boolean or binary data variables where each variables can take on only the values 1 (true) and 0 (false). One simple application of such a data may be the cases where we manipulate bits of a data item. For example, in floating-point addition we need to shift mantissa bits.

5.3 Types of Instructions

Instructions in computer are used to translate high level language programs. There are many ways of classifying the instructions. One of them depends on the number of operands explicitly specified in the instructions. They are zero single double and three operand instructions.

Zero Operand Instructions

The instructions in which operands are not explicitly specified are known as zero-operand or address instructions. The implicit operands are assumed to be in the registers.

Examples

CMC	Complement carry
STC	Set carry
CLD	Clean direction flag

The above instruction operate on implicitly assumed flag register.

One Operand Instructions

The instructions in which a single operand is explicitly specified are known as one operand or address instructions.

Examples

INC A X	AX $\leftarrow$ AX + 1
DEC CX	CX $\leftarrow$ CX –1 (decrement CX content by 1)
POP BX	POP BX from the stack

The above instructions operate n the single operand specified in the instruction.

Two Operands Instruction

The instruction in which two operands are explicitly specified are known as two operands or address instructions.

Examples

MOV AX, 100	AX ← 100
ADD AX, BX	AX ← AX + BX
SUB CX, 1	CX ← CX –1

Three operands Instructions

The instructions in which three operands are explicitly specified are known as three operands or address instructions.

Examples

IMUL BX, CX, 10	BX ← CX ✶10
IMUL AX, BX, 1024	AX ← BX ✶ 1024

The three operands instructions IMUL is supported by the 80186 processors only.

The processors also supports a wide array of instructions to perform the movement of data (between memory, CPU, I/o devices), arithmetic and logical operations(addition, subtraction, multiplication, division, data, conversion, comparisons), branch (jump instructions), and control of processor operations (enable interrupt, set direction flag), string operations and protection control. The figure -3 below shows classification of instruction.

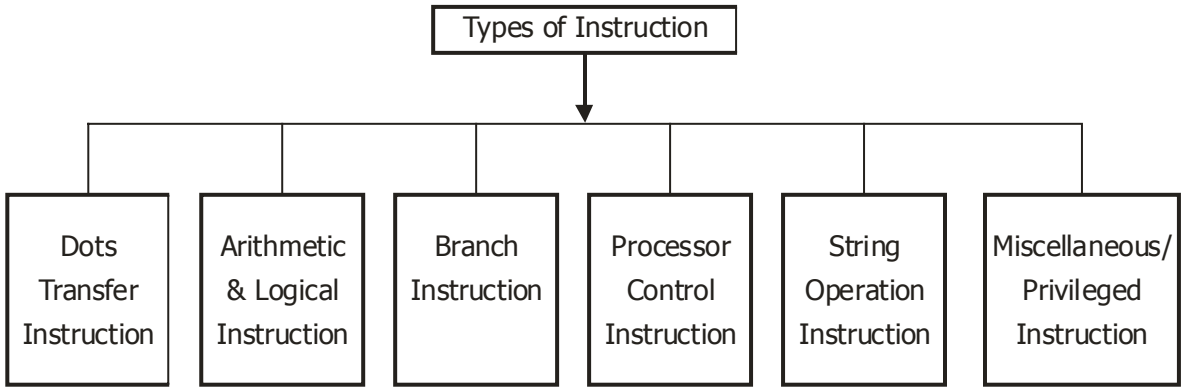


Figure 3 : Types of Instruction

Data transfer Instructions

These instructions transfer data from one location in the computer to another location without changing the data contents.

The most common transfer are between.

- Resister to register.
- Memory to register.

- Register to memory.
- Register to I/O part and vice versa.

These instruction have the following properties.

- Two operands, the source and destination
- They must be of the same data-type that is either of type byte or type word.
- Both the source and destination control refer to memory locations in the same instructions.
- The source can be register or a memory location or an immediate data.
- The destination can be register or a memory location.
- These instructions do not effect the CPU flags.

The instruction also need the mode of addressing for each operand. A table is given below, which lists some data transfer instructions with their mnemonic symbols. Different computers may use different mnemonics for the same instruction.

Operation Name	Mnemonic	Description
MOVE	MOV	Transfer data from Register/Memory/ Immediate to Register/Memory.
XCHG	XCHG	Exchange the contents of the source and destination.
PUSH	PUSH	Push register or memory to stack
POP	POP	Transfer data from top of stack to processor register
LOAD	LD	Loads the contents from memory to register
Set	SET	Specified operands replaced by 1.
Clear	CLEAR	Specified operand replaced by 0's.

Arithmetic and Logic Instructions

These instructions perform arithmetic and logical operations on data.

Arithmetic : The basic arithmetic operands are ADD, SUBTRACT, MULTIPLY and DIVIDE.

Following points are to be noted about arithmetic operations:

- Instruction generally involves two operands that is source and destination.
- They must be of same data type.
- Instructions affect the CPU flag to reflect the status of operations.

Some arithmetic operation their mnemonics and description are given below in the table.

Arithmetic operations	Mnemonics	Description
Addition	ADD	Adds two operands and stores the result.
Subtraction	SUB	Subtracts source from destination.
Decrement	DEC	Subtract one from destination.
Compare	CMP	Compare source with destination and flags are set to indicate carry.
Multiply	MUL	Multiplies source to destination.
Divide	DIV	Divide accumulator by unsigned value stored in reg/men.

Logical Instruction: AND, OR, NOT, XOR are same logical instructions that operates on binary data stored in register. Logical shift (Left shift or Right shift) is also used for transfer of bits either to the left or to the right.

Example:

(i) AND AX, FOOOH, if AX = FFOOH
then AX = FFOO & FOOO
= FOOOH

(ii) R_1 = 1101 1001
R_2 = 0010 1110
R x or R_2 = 1111 0111

Shift Operation : It is basically of three types :

(i) Logical shift : Insets zero to the end of bit position and the other bits of a word are shifted left or right respectively.

(ii) Arithmetic shift: On the arithmetic shift right, the sign bit is replicated into the bit position to its right. On an arithmetic shift left, a logical shift left is performed on all bits but the sign bit, which is retained.

(iii) Circular shift: Rotate left or rotate right. Bits are shifted out at one end of the word are not lost as in a logical shift but are calculated back into the other end.

Branch Instructions

The branch instructions transfers control from the normal sequence of instruction execution to the specified destination or target instruction. These instructions are broadly categorized as,

(i) Conditional branch and

(ii) Unconditional branch.

The conditional jump instructions transfer control to the target location if some specified condition is met or satisfied. some conditional branch jump instructions are JA, JB, JE etc.

Example:

Branch Operation	Mnemonic	Description
Jump	JMP	Unconditional transfer of control to the table.
Jump on Above	JA	Jump if (CF = 0 and ZF = 0) after unsigned math.
Looping	Loop	Loop if CX ≠ zero

Note : CF = carry flog; ZF = Zero flag and CX = Register.

Processor control instructions

The processor supports a variety of instructions modifying the CPU status flag register called as the processor control instructions. Almost all the instructions except those in the data transfer category uses the status of the flags in the flag register during the execution. Generally the flag register is set to reflect the status of the result automatically. Most of the processor control instructions affect the CPU flags.

Examples:

Control Operation	Mnemonic	Description
Clear carry	CLC	Clears the carry flag
Complement carry	CMC	Complements the carry flag
Set carry	STC	Set the carry flog to 1.

String Operations Instructions

A string is a contiguous sequence of types or words. strings can be used to hold any type of data or information that will fit into bytes. There are number of operations that can be performed with strings.

All string instructions require two operands. All instructions assume that the same source operand is in the data segment (DS) and the destination is in the extra segment (ES). In string instructions, the operands are not mentioned explicitly. Therefore, all the registers used by the instructions must be loaded prior to the execution of the instruction.

Example:

Shang Operation	Mnemonics	Description
Clear direction flag	CLD	Clear direction flag so that pointers in shing instructions are updated by incrementing.

Move string	MOVS	Transfers a byte or a word from the source string to the destination string.

The **SKIP** instruction is a zero-address instruction and skips the next instruction to be executed in sequence. In other words, it increments the value of PC by one instruction length. The **SKIP** can also be conditional. For example, the instruction **ISZ** skips the next instruction only if the result of the most recent operation is zero.

CALL and **RETN** are use for CALLing subprograms and RETurning from them. Assume that a memory stack has been built such that stack pointer points to a non-empty location stack and expand towards zero address.

Miscellaneous and Privileged Instructions: These instructions do not fit in any of the above categories. I/O instructions: start I/O, stop I/O, and test I/O. typically, I/O destination is specified as an address. Interrupt and state-swapping operations: There are two kinds of exceptions, interrupts that are generated by hardware and traps, which are generated by programs. Upon receiving interrupts, the state of current processes will be saved so that they can be restarted after the interrupts has been taken care of. Most computer instructions are divided into two categories, privileged and non-privileged. A process running in privileged mode can execute all instructions from the instruction set while a process running in user mode can only execute a sub-set of the instructions. I/O instructions are one example of privileged instruction, clock interrupts are another one.

5.3.3 Number of Addresses in an Instruction

The Instruction set Architecture (ISA) of a processor can be differentiated using five categories:

- Operand storage in the CPU.
- Number of explicitly named operands.
- Operand location.
- Operations.
- Type and size of operands.

In this section we will look into some of the architectures that are common in contemporary computer. But before we discuss the architecture, let us look into some basic characteristics of instruction set.

- The operands can be addressed in memory, registers or I/O device address.
- Instruction having less number of operand addresses in an instruction may require lesser bits in the instruction; however, it also restricts the range of

functionality that can be performed by the instructions. This implies that a CPU instruction set having less number of addresses has longer programs, which means longer instruction execution time. On the other hand, having more addresses may lead to more complex decoding and processing circuits.

- Most of the instructions do not require more than three operand addresses. Instructions having fewer addresses than three, use register implicitly for operand reference can result in smaller instructions as only few bits are needed for register addresses as against memory addresses.
- The type of internal storage of operands in the CPU is the most basic differentiation.
- The three most common types of IS As are;

1. **Evaluation stack :** The operand are implicitly on the top of the stack.
2. **Accumulator :** One operand is implicitly the accumulator.
3. **General Purpose Architecture (GPR):** All operands are explicit, either registers or memory location.

Evaluation Stack Architecture : A stack is a data structure that implements Last-In-First-Out (LIFO) access policy. We could add an entry to the stack with a PUSH (value) and remove an entry from the stack with a POP (). No explicit operands are there in ALU instructions, but one in PUSH/POP. Example of such computers are Burroughs B5500/6500, HP 3000/70etc.

ON a stack machine "C = A+B" might be implemented as:

```
PUSH A
PUSH B
ADD // operator POP operand(s) and PUSH result(s) (implicit on top of stack)
POP C
```

Stack Architecture : Pros and Cons

- Small instructions (do not need many bits to specify the operation).
- Compiler is easy to write.
- Lots of memory accesses required-everything that is not on the stack is in memory. Thus, the machine performance is poor.

Accumulator Architecture: An accumulator is a specially designated register that supplies one instruction operand and receives the result. The instructions in such machines are normally one-address instructions. The most popular early architectures were IBM 7090, DEC PDP-8 etc.

LOAD A // Load memory location A into accumulator

ADD B // Add memory location B to accumulator

STORE C // Store accumulator value into memory location C

Accumulator Architecture: Pros and CONS

- Implicit use of accumulator saves instruction bits
- Result is ready for immediate reuse, but has to be saved in memory if next computation does not use it right away.
- More memory accesses required than stack.

General Purpose Register (GPR) Architecture: A register is a word of internal memory like the accumulator. GPR architecture is an extension of the accumulator idea, i.e., use a set of general-purpose registers, which must be explicitly named by the instruction. Registers can be used for anything either holding operands for operations or temporary intermediate value. The dominant architectures are IBM 370, PDP-11 and all Reduced Instant set Computer (RISC) machines etc. The major instruction set characteristic whether an ALU instruction has two or more operands divides GPR architectures:

"C = A + B" might be implemented on both the architectures as:

Register - Memory	**Load/Store through Registers**
LOAD RI, A	LOAD R1, A
ADD RI, B	LOAD R2, B
STORE C, RI	Add R3, R1, R2
	STORE C, R3

General Purpose Register architecture: Pros and Cons

- Register can be used to store variable as it reduces memory traffic and speeds up execution. It also improves code density, as register names are shorter than memory address.
- Instruction must include bits to specify which register to operate on, hence large instruction size than accumulator type machines.
- Memory access can be minimized (register can hold lots of intermediate values).
- Implementation is complicated, as complier writer has to attempt to maximize register usage.

Early machines used stack or accumulator architectures but in the last 15 year all CPUs are made up of GPR processors. The three major reasons are:

(i) that register are faster than memory;

(ii) the more data that can be kept internally in the CPU the faster the program will run.

(iii) that register are easier for a compiler to use.

But while CPU's with GPR were clearly better than previous stack and accumulator based CPU's yet they were lacking in several areas being: Instructions were of varying length from 1 bye to 6-8 bytes. This causes problems with the pre-fetching and pipelining of instructions. ALU instructions could have operands that were memory locations because the time to access memory is slower and so does the whole instruction.

Thus in the early 1980 the idea of **RISC** was introduced. RISC stand for reduced instruction set computer. Unlike CISC, this ISA uses fewer instructions with simple constructs so they can be executed much faster within the CPU without having to use memory as often. The first RISC CPU, the MIPS 2000, has 32 GPRs. MIPS is a load/store architecture, which means that only load and store instructions access memory. All other computational instructions operate only on values stored in stored registers.

5.4 Addressing Schemes

Addressing scheme is defined as a method of specifying the effective address of the operands in memory. Whenever the microprocessor executes instructions, it performs a very specific function on data. These data terms often referred to as operands, may be part of the instruction, may be residing in one of the internal registers of the microprocessor or may be stored at a specific address in memory. To access these different types of operands, the microprocessor is provided with various addressing mode. The addressing mode increases the flexibility of the programming language and is useful in implementing the constructs and data structures of the powerful high level programming languages.

5.4.1 Types of Addressing Schemes

We will describe the different addressing schemes with reference to 8086 architecture. Most of the machines employ a set of addressing modes. We will describe some very common addressing modes used in most of the machines. A tree of common addressing mode is given in figure below:

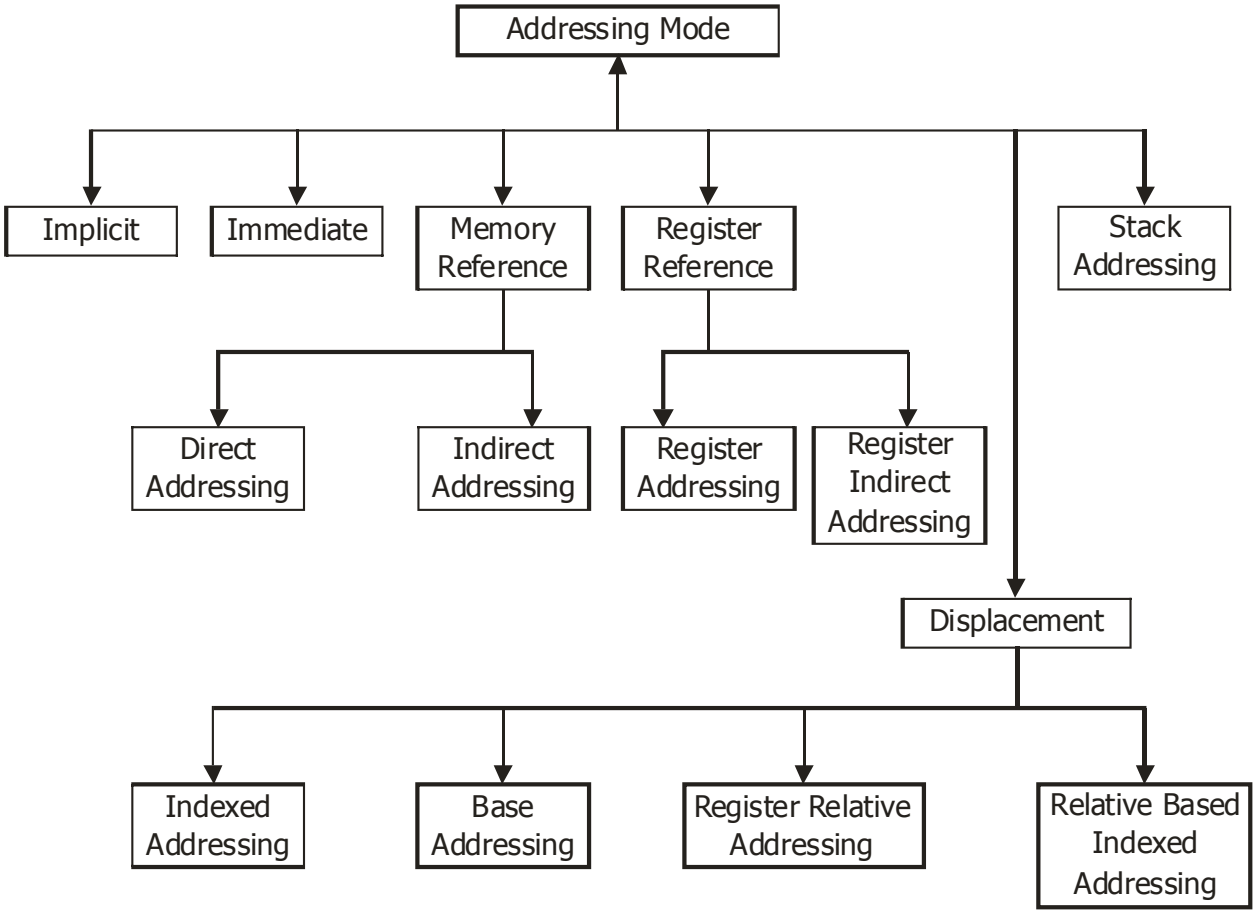


Figure 4 : Addressing Modes

The three components used in accessing the memory operands are:

(i) Base registers: BX and BP

(ii) Index Register : SI and DI

(iii) Displacement : 8 bit sign extended or 16 bit value.

The base, index and displacement can be combined as,

BX SI

or + or + Displacement (disp) = memory address of the operand.

BP DI

(Base)

We will use MOV instruction to explain the different addressing modes. The format of MOV is :

MOV dest, source. {It move the content of the source to destination}

5.4.1.1 Implicit Addressing Mode :

In this mode, the data is assumed to be are defined in the instruction. For example, in the statements CMC, STD or CLC, etc. the data is in the flag register. Such instructions are called zero address instruction.

5.4.1.2 Immediate Addressing Mode

In this mode data is the part of the instruction. It does not involve computation of address.

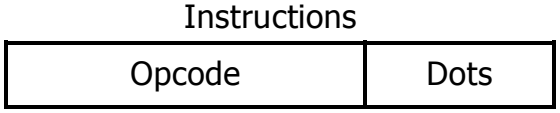


Figure 5 : Immediate addressing

Example :

MOV AX, 100 ; AX ← 100

The data 100 is the part of the instructions. The value 100 is moved into the register AX.

Some important points of immediate addressing :

(i) Used for initialing the value of a variable.

(ii) No additional memory access required for executing the instructions.

(iii) The size of the instruction and operand field limited.

5.4.1.3 Direct Addressing Mode

In this mode the address of the data is a part of the instructions.

Figure 7 : Direct Addressing

For example :

MOV AX, [1050H]

The data is in the memory location pointed by the address 1050H. Specified constant value gives the offset.

Important point to remember :

(i) This addressing scheme facilitates global variable with fixed offset values to be addressed directly.

(ii) Provides a limited space for address because if the address field has in bits then memory space would be 2^n memory location.

(iii) Only one memory reference is required to fetch the operand.

5.4.1.4 Indirect Addressing

In this mode, the address of the operand is in the register. The data on which the instruction operates is pointed to by the contents of the register which is a part of the instruction. It requires two memory reference.

For example :

MOV AX, [BX]

The content of register BX is copied to AX.

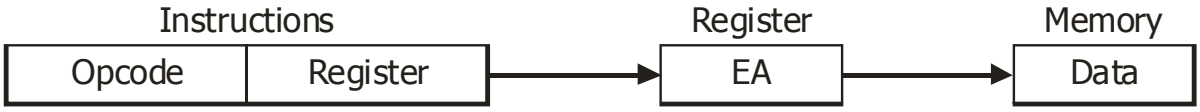


Figure 6 : Indirect mode/Register indirect

5.4.1.5 Register Addressing Mode

In this mode the operands on which instructions operate are stored in the specified registers. For a 16-bit operand, a register may be AX, BX, CX, DX, SI, DI, SP and BP and for 8-bit operand, a register may be AH, AL, BH, BL, CH, CL, and DH, DL.

Examples :

MOV AX, DX ; AX ← DX

MOV BX, AX ; BX ← AX

SUB BP, CX ; BP ← BP — CX

It is similar to direct addressing except that the register name or number is substituted for memory address.

Points to remember :

(i) Register access is faster than memory access results in faster instructions execution.

(ii) The size of register is smaller than memory address. This reduces the instruction size.

Figure 8 : Register Addressing

5.4.1.6 Register Indirect Mode

The effective address (EA) of the data is in the base register BX or in index register SI and DI that is specified in the instruction.

That is

$$EA = \begin{matrix}(BX)\\(SI)\\(DI)\end{matrix}$$

In the instruction EA is designated as [Register].

Examples :

MOV AX, [BX]; copy BX content to AX.

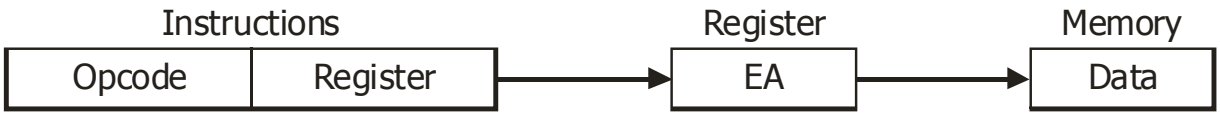


Figure 9 : Register Indirect

5.4.1.7 Indexed Addressing Mode

In this mode the operand field of the instruction contains an address and an index register, which contains an offset. This addressing is used to address the consecutive location of memory.

For example :

MOV DX, [SI] + 5H

Here the physical address (PA) obtained

as $PA = \begin{Bmatrix}SI\\DI\end{Bmatrix} + \begin{Bmatrix}8\text{-bit displacement}\\16\text{-bit displacement.}\end{Bmatrix}$

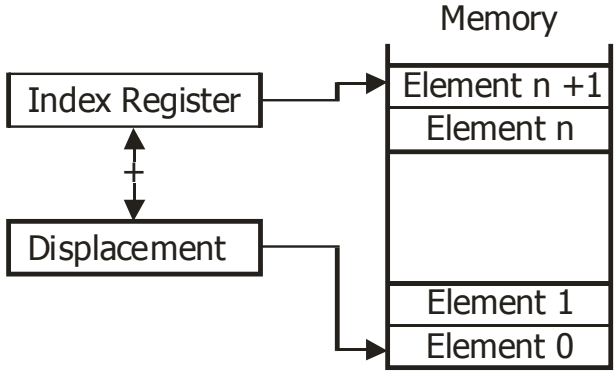


Figure 10 : Indexed Addressing

5.4.1.8 Base Addressing

In this mode effective address of the operand is obtained by addressing a direct or indirect displacement to the content of either base register BX or base pointer BP. The physical address (PA) can be calculated as :

$$PA = \begin{Bmatrix} BX \\ BP \end{Bmatrix} + \begin{Bmatrix} 8\text{-bit displacement} \\ 16\text{-bit displacement.} \end{Bmatrix}$$

For example :

```
MOV   AX, [BX]
MOV   CX, [BX] + 10H
MOV   AL, [BP] + 5H
```

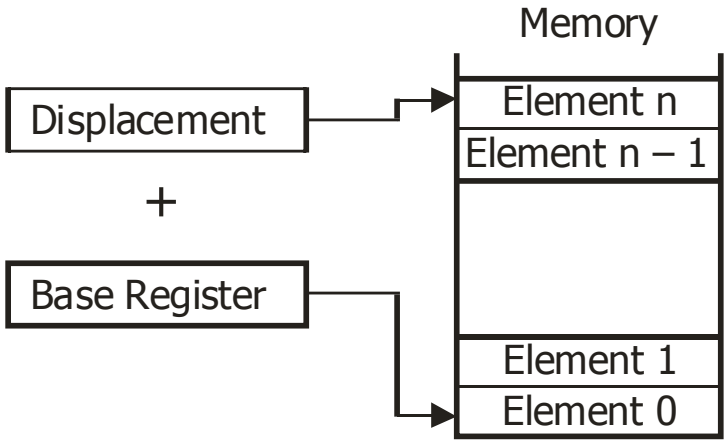


Figure 11 : Base Addressing

5.4.1.9 Register Relative Mode

In this mode the effective address (EA) is generated by adding the content of the base register or index register to an 8-bit or a 16-bit displacement value that is,

```
EA  =  (BX) + disp
       (BP) + disp
       (SI) + displ
       (DI) + displ
```

Examples

(1) A [BX]

(2) [BX + 20]

The process of register relative mode can be shown as follows :

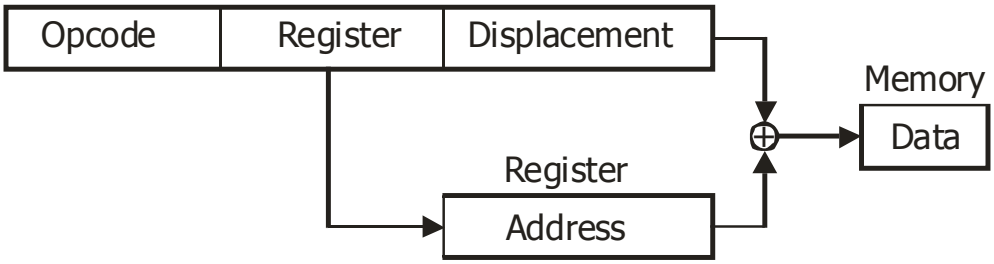


Figure 12 : Register relative mode

5.4.1.10 Relative Based Indexed Mode

The effective address (EA) is computed by addressing the contents of the base register and index register with an 8-bit or a 16-bit displacement value that is :

EA = (BX) + (SI) + disp
(BX) + (DI) + disp
(BP) + (SI) + disp
(BP) + (DI) + disp

Examples

(1) [BP + 2] [SI + 1]

(2) A [BX] [SI]

(3) X-ARRAY [BP] [SI + 2]

The process of relative based indexed addressing mode is shown below in the figure.

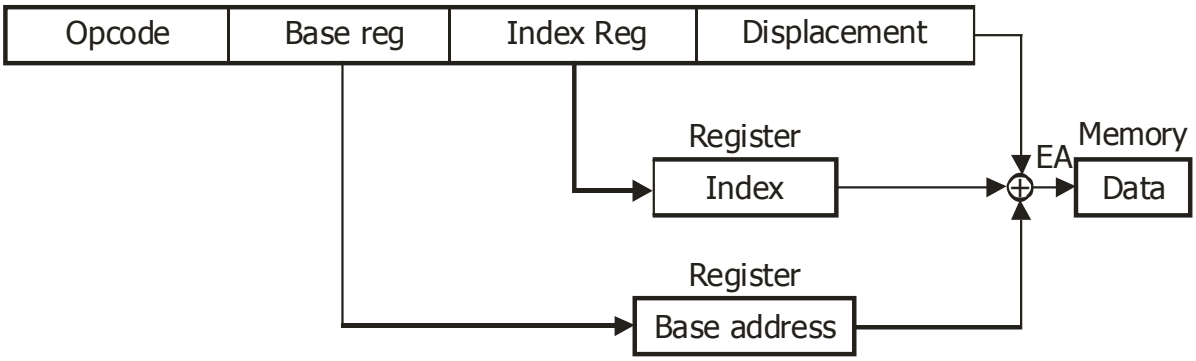


Figure 13 : Relative based index addressing

5.4.1.11 Stack Addressing

In this addressing scheme, the operand is implied as top of stack. It is not explicit, but implied. It uses a CPU Register called Stack Pointer (SP). The SP points to the top of the stack i.e. to the memory location where the last value was

pushed. A stack provides a sort-of indirect addressing and indexed addressing. This is not a very common addressing scheme. The operand is found on the top of a stack. In some machines the top two elements of stack and top of stack pointer is kept in the CPU registers, while the rest of the elements may reside in the memory. Figure 14 shows the stack addressing schemes.

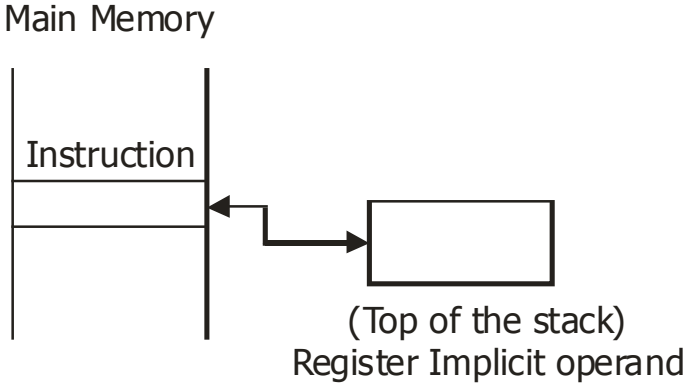


Figure 14 : Stack Addressing

5.5 Instruction Set and Format Design Issues

Some of the basic issues of concerns for instruction set design are :

Completeness : The primary concern is that the instruction set should be complete which means there is no missing functionality. It should include instructions for the basic operations that can be used for creating any possible execution and control operation.

Orthogonal : The secondary concern is that the instructions be orthogonal, that is, it should not be redundant. For example, different addressing modes may be redundant when there are more instructions used than required. Redundancy results into longer CPU time.

An instruction format is used to define the layout of the bits allocated to these elements of instructions. In addition, the instruction format explicitly or implicitly indicates the addressing modes used for each operand in that instruction. Designing of instruction format is a complex art. In this section, we will discuss about the design issues for instruction sets of the machines briefly.

5.5.1 Instruction Length

Significance : It is the basic issue of the format design. It determines the richness and flexibility of a machine.

Basic Trade-off : Smaller instruction (less space) Versus desire for more powerful instruction repertoire.

Normally programmer desire :

- **More op-code and operands :** as it results in smaller programs
- **More addressing modes :** for greater flexibility in implementing functions like table manipulations, multiple branching.

However, a 32 bit instruction although will occupy double the space and can be fetched at double the rate of a 16 bit instruction, but can not be doubly useful.

The important factors, which must be considered for deciding about instruction length are given below in the table :

Factors	Descriptions
Memory size	If larger memory range is to be addressed, then more bits may be required in address field.
Memory organization	If the addressed memory is virtual memory then memory range which is to be addressed by the instruction is larger than physical memory size.
Memory transfer length	Instruction length should normally be equal to data bus length or multiple of it.
Memory transfer	The data transfer rate from the memory ideally should be equivalent to the processor speed. It can become a bottleneck if processor executes instructions faster than the rate of fetching the instructions. One solution for such problem is to use cache memory or another solution can be to keep instruction short.

Normally an instruction length is kept as a multiple of length of a character (that is 8 bits), and equal to the length of fixed-point number. The term word is often used in this context. Usually the word size is equal to the length of fixed point number or equal to memory-transfer size. In addition, a word should store integral number of characters. Thus, word size of 16 bit, 32 bit, 64 bit are becoming very common and hence the similar length of instructions are normally being used.

5.5.2 Allocation of Bits Among Opcode and Operand

The trade-off here is between the numbers of bits of opcode versus the addressing capabilities. An interesting development in this regard is the development of variable length opcode.

Some basic factors that are considered for selection of addressing bits are given below in the table :

Factors	Descriptions
Number of addressing modes	The more are the explicit addressing modes the more bits are needed for mode selection. However, some machines have implicit modes of addressing.
Number of operands	Fewer number of operand reference in an instruction although require less bit yet result in longer programs. Today machines have two operand reference in an instruction. Each of these registers may need a addressing mode indicator field.
Register addressing versus memory addresses	The register reference require fewer bit in comparison to the memory addresses. In general, the number of user visible registers provided is 16 to 32. Some of these registers may be used for special purposes.
Granularity of address :	As far as memory references are concerned, granularity implies whether an address is referencing a byte or a word at a time. This is more relevant for machines, which have 16 bits, 32 bits and higher bits words. Byte addressing although may be better for character manipulation, however, requires more bits in an address. For example, memory of 4K words (1 word = 16 bit) is to be addressed directly then it requires : WORD Addressing = 4K words $= 2^{12}$ words $\Rightarrow$ 12 bits are required for word addressing. Byte Addressing $= 2^{12}$ words $= 2^{13}$ bytes $\Rightarrow$ 13 bits are required for byte addressing.

5.5.3 Variable Length of Instruction

With the better understanding of computer instruction sets, the designers came up with the idea of having a variety of instruction formats of different length. The advantages of such a scheme are :

- Large number of operations can be provided which have different lengths of instructions.
- Flexibility in addressing scheme can be provided efficiently and compactly.

However, the basic disadvantage of such a schemes is to have a complex CPU.

An important aspect about these variables length instructions is : "The CPU is not aware about the length of next instruction which is to be fetched". This problem can be handled if each instruction fetch is made equal to the size of the longest instruction. Thus, sometimes in a single fetch multiple instructions can be fetched.

5.6 Example of Instruction Format

MIPS is an acronym for **Microprocessor without Interlocked Pipeline Stages.** It is a microprocessor architecture developed by MIPS Computer Systems Inc. most widely known for developing the MIPS architecture. The MIPS CPU family was one of the most successful and flexible CPU designs throughout the 1990s. MIPS is implementation of a RISC architecture.

MIPS R2000 ISA :

- Are Designed for use with high-level programming languages and have small set of instructions and addressing modes, easy to compile.
- Minimize/balance amount of work (computation and data flow) per instruction
- Allows for parallel execution
- Load-store machine
- Large register set, minimize main memory access
- Fixed instruction width (32-bits), small set of uniform instruction encodings
- Minimize control complexity, allow for more registers

MIPS instruction fall into 5 classes :

— Arithmetic/logical/shift/comparison

— Control instructions (branch and jump)

— Load/store

— Other (exception, register movement to/from GP registers, etc.)

The MIPS CPU has a five-stage CPU pipeline to execute multiple instructions at the same time. It defines the 5 steps of execution of instructions that may be performed in an overlapped fashion. The figure below show the pipelining concept :

Instruction execution stages

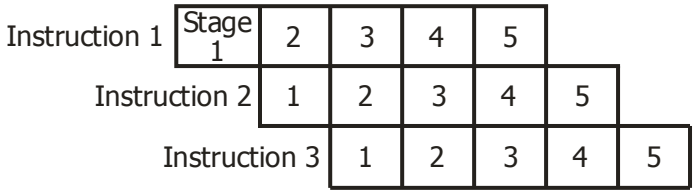


Figure 15 : Pipeline

From the above figure we have that :

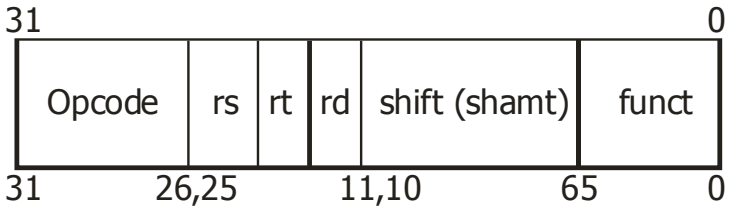
- All the stages are independent and distinct, that is, the second stage execution of Instruction should not hinder Instruction 2.
- The overall efficiency of the system becomes better.

The early MIPS architectures had 32-bit instructions and later versions have 64-bit implementations.

The first commercial MIPS CPU model, the **R2000**, whose instruction format is discussed below, has thirty-two 32-bit registers and its instructions are 32 bit long.

R Format

Converting an R mnemonic into the equivalent binary machine code is performed in the following way :



Opcode–The opcode is the machine code representation of the instruction mnemonic. Several related instructions can have the same opcode. The opcode field is 6 bits long (bit 26 to bit 31).

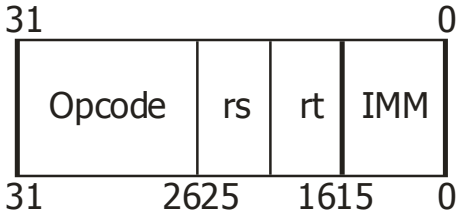
rs, rt, rd–The numeric representations of the source registers and the destination register. These numbers correspond to the $X representation of a register, such as $0 or $31. Each of these fields is 5 bits long. (25 to 21, 20 to 16, and 15 to 11, respectively). Interestingly, rather than rs and rt being named r1 and r2 (for source register 1 and 2), the registers were named "rs" and "rt" because t comes after s in the alphabet. This was most likely done to reduce numerical confusion.

Shift (shamt)–Used with the shift and rotate instructions, this is the amount by which the source operand *rs* is rotate/shifted. This field is 5 bit long (6to 10).

Funct–For instructions that share an opcode, the **funct** parameter contains the necessary control codes to differentiate the different instructions. 6 bits long (0 to 5). Example : Opcode 0 × 00 accesses the ALU, and the funct selects which ALU function to use.

I Format

I instructions are used when the instruction must operate on an immediate value and a register value. Immediate values may be a maximum of 16 bits long. Larger numbers may not be manipulated by immediate instructions. I instruction are converted into machine code words in the following format.



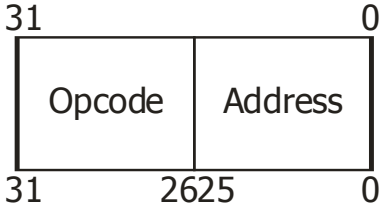
Opcode–The 6 bit opcode of the instruction. In I instructions, all mneumonics have a one-to-one correspondence with the underlying opcodes. This is because there is no **funct** parameter to differentiate instructions with an identical opcode. 6 bits (26 to 31)

rs, rt– The source and destination register operands, respectively. 5 bits each (21 to 25 and 16 to 20, respectively).

IMM– The 16 bit immediate value. 16 bits (0 to 15). This value is usually used as the offset value in various instructions, and depending on the instruction, may be expressed in two's complement.

J Format

J instructions are used when a jump needs to be performed. The J instruction has the most space for an immediate value, because addresses are large number. J instructions have the following machine-code format :



Opcode-The 6 bit opcode corresponding to the particular jump command. (26 to 31).

Address-A 26-bit address of the destination. (0 to 25).

All MIPS instructions are of the same length, requiring different kinds of instruction formats for different types of instructions.

MIPS uses various addressing modes :

1. Register and Immediate addressing modes for operations.
2. Immediate and Displacement addressing for Load and Store instructions.

(In displacement addressing, the operand is at the memory location whose address is the sum of a register).

5.7 Summary

In this unit we have explained Instruction Set Architecture (ISA), its characteristics, design confederation etc. We also discussed different types of (ISA), types of instructions and the operation performed by them. Various addressing schemes have also been discussed with examples. An example of instruction format has also been provided to make the concept more clear.

5.8 Questions

1. What is an Instruction ? What are the elements of an instruction ?
2. Explain in brief the design considerations of an instruction set.
3. What are different operand Data Types ?
4. Explain General Purpose Register (GPR).
5. What is addressing scheme ? Why it is required ?
6. Describe different types of addressing schemes.
7. What are the factors that must be considered for deciding the instruction length ?
8. Explain some factors considered for selection of addressing bits.
9. What are the advantages of variable length instruction ?
10. Give and describe an example of an instruction format.

5.9 Suggested Readings

1. Microprocessor X86 programming by K.R. Venugopal and Raj Kumar.
2. Computer Architecture and organization by John. P. Hayer.

● ● ●